

Name : Venna Meenakshi Reddy

Register Number : 192471048

Brute Force - Geometric & Exhaustive Search Problems

Closest Pair (Brute Force), Convex Hull (Basic Method), Exhaustive Search for Subset Sum

Dr. Kalaivani P

Topic 1: Closest Pair of Points (Brute Force Approach)

1. Write a program that finds the closest pair of points in a set of 2D points using the brute force approach.

Input:

- A list of points represented by coordinates (x, y).

Points: [(1, 2), (4, 5), (7, 8), (3, 1)]

Output:

- The two points with the minimum distance.
- The minimum distance between them.

Example Output:

Closest pair: (1, 2) - (3, 1)

Minimum distance: 1.4142135623730951

2. Write a program to find the closest pair of points among the given set of coordinates using exhaustive comparison of all pairs.

Input:

Points: [(2, 3), (5, 4), (1, 7), (8, 9), (4, 6)]

Output:

Closest pair: (2, 3) - (5, 4)

Minimum distance: 3.1622776601683795

3. Develop a program to calculate the shortest distance between any two points using the brute force closest pair algorithm.

Input:

Points: [(0,0), (2,2), (3,1), (6,5)]

Output:

Closest pair: (2,2) - (3,1)

Minimum distance: 1.4142135623730951

4. Write a program to identify the closest pair of points from a given collection of 2D points using brute force technique.

Input:

Points: [(10,20), (15,25), (30,40), (12,22)]

Output:

Closest pair: (10,20) - (12,22)

Minimum distance: 2.8284271247461903

5. Write a program to find the closest pair of points in a given set using the brute force approach. Analyze the time complexity of your implementation. Define a function to calculate the Euclidean distance between two points. Implement a function to find the closest pair of points using the brute force method. Test your program with a sample set of points and verify the correctness of your results. Analyze the time complexity of your implementation. Write a brute-force algorithm to solve the convex hull problem for the following set S of points? P1 (10,0)P2 (11,5)P3 (5, 3)P4 (9, 3.5)P5 (15, 3)P6 (12.5, 7)P7(6, 6.5)P8 (7.5, 4.5).How do you modify your brute force algorithm to handle multiple points that are lying on the same line?

Given points: P1 (10,0), P2 (11,5), P3 (5, 3), P4 (9, 3.5), P5 (15, 3), P6 (12.5, 7), P7 (6, 6.5), P8 (7.5, 4.5).

output: P3, P4, P6, P5, P7, P1

Topic 2: Convex Hull (Basic Method)

6. Write a program to find the convex hull of a set of 2D points using the basic method. Input:

Points: [(0,0), (1,1), (2,2), (0,2), (2,0)]

Output:

Convex Hull Points:

(0,0), (2,0), (2,2), (0,2)

7. Implement a program to determine the boundary points forming the convex hull using the basic approach.

Input:

Points: [(1,2), (3,1), (5,3), (4,6), (2,5)]

Output:

Convex Hull Points:

(1,2), (3,1), (5,3), (4,6), (2,5)

8. Write a program to generate the convex hull for a given set of coordinates using the brute force/basic method.

Input:

Points: [(0,3), (1,1), (2,2), (4,4), (3,0)]

Output:

Convex Hull Points:

(3,0), (4,4), (0,3)

9. Develop a program to find the outer boundary of a set of points using the convex hull algorithm.

Input:

Points: [(2,2), (4,1), (6,3), (5,5), (3,6), (1,4)]

Output:

Convex Hull Points:

(4,1), (6,3), (5,5), (3,6), (1,4), (2,2)

10. Write a program to calculate the convex hull of given points and display the points forming the polygon boundary.

Input:

Points: [(1,1), (2,4), (5,5), (6,2), (3,0)]

Output:

Convex Hull Points:

(3,0), (6,2), (5,5), (2,4)

Topic 3: Exhaustive Search for Subset Sum

11. Write a program to find whether a subset exists with a given sum using exhaustive search technique.

Input:

Set: [3, 34, 4, 12, 5, 2]

Target Sum: 9

Output:

Subset found: [4,5]

Sum: 9

12. Include test cases with different city configurations to demonstrate the program's functionality. Print the shortest distance and the corresponding path for each test case.

Test Cases:

1. Simple Case: Four cities with basic coordinates (e.g., [(1, 2), (4, 5), (7, 1), (3, 6)])
2. More Complex Case: Five cities with more intricate coordinates (e.g., [(2, 4), (8, 1), (1, 7), (6, 3), (5, 9)])

Output:

Test Case 1:

Shortest Distance: 7.0710678118654755

Shortest Path: [(1, 2), (4, 5), (7, 1), (3, 6), (1, 2)]

Test Case 2:

Shortest Distance: 14.142135623730951

Shortest Path: [(2, 4), (1, 7), (6, 3), (5, 9), (8, 1), (2, 4)]

13. You are given a cost matrix where each element $\text{cost}[i][j]$ represents the cost of assigning worker i to task j . Develop a program that utilizes exhaustive search to solve the assignment problem. The program should Define a function `total_cost(assignment, cost_matrix)` that takes an assignment (list representing worker-task pairings) and the cost matrix as input. It iterates through the assignment and calculates the total cost by summing the corresponding costs from the cost matrix Implement a function `assignment_problem(cost_matrix)` that takes the cost matrix as input and performs the following Generate all possible permutations of worker indices (excluding repetitions). **Test Cases:**

Input

1. **Simple Case:** Cost Matrix: [[3, 10, 7], [8, 5, 12], [4, 6, 9]]
2. **More Complex Case:** Cost Matrix: [[15, 9, 4], [8, 7, 18], [6, 12, 11]]

Output:

Test Case 1:

Optimal Assignment: [(worker 1, task 2), (worker 2, task 1), (worker 3, task 3)]

Total Cost: 19

Test Case 2:

Optimal Assignment: [(worker 1, task 3), (worker 2, task 1), (worker 3, task 2)]

Total Cost: 24

14. You are given a list of items with their weights and values. Develop a program that utilizes exhaustive search to solve the 0-1 Knapsack Problem. The program should:

1. Define a function `total_value(items, values)` that takes a list of selected items (represented by their indices) and the value list as input. It iterates through the selected items and calculates the total value by summing the corresponding values from the value list.
2. Define a function `is_feasible(items, weights, capacity)` that takes a list of selected items (represented by their indices), the weight list, and the knapsack capacity as input. It checks if the total weight of the selected items exceeds the capacity.

Test Cases:

1. Simple Case:

- Items: 3 (represented by indices 0, 1, 2)
- Weights: [2, 3, 1]
- Values: [4, 5, 3]
- Capacity: 4

2. More Complex Case:

- Items: 4 (represented by indices 0, 1, 2, 3)
- Weights: [1, 2, 3, 4]
- Values: [2, 4, 6, 3]
- Capacity: 6

Output:

Test Case 1:

Optimal Selection: [0, 2] (Items with indices 0 and 2)

Total Value: 7

Test Case 2:

Optimal Selection: [0, 1, 2] (Items with indices 0, 1, and 2)

Total Value: 10

15. Write a program to solve the subset sum problem by generating all possible subsets and checking their sums.

Input:

Set: [15,10,12,7,5]

Target Sum: 22

Output:

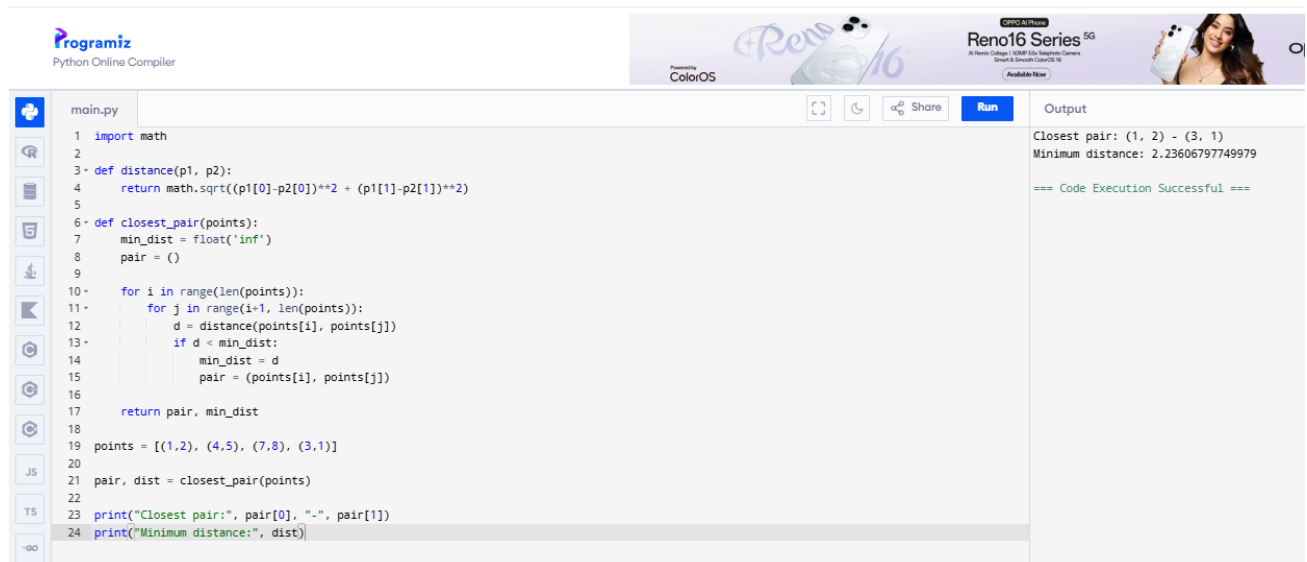
Subset found: [15,7]

Sum: 22

OUTPUTS:

Topic 1: Closest Pair of Points (Brute Force Approach)

1. Write a program that finds the closest pair of points in a set of 2D points using the brute force approach.



The screenshot shows a Python Online Compiler interface. The code in the editor is as follows:

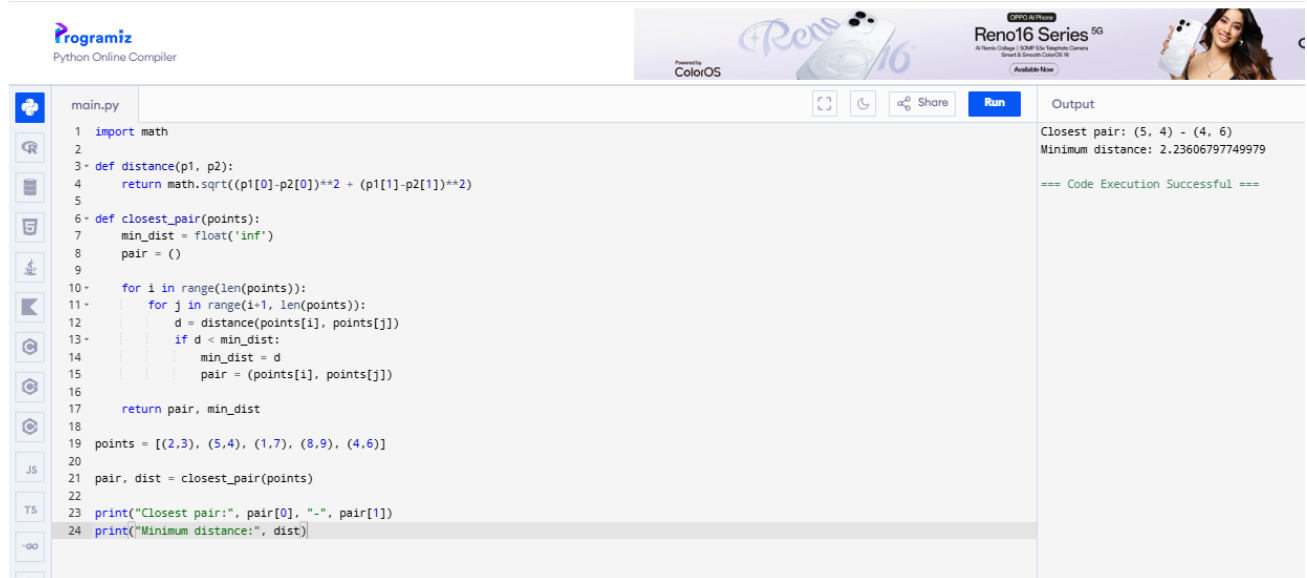
```
1 import math
2
3 def distance(p1, p2):
4     return math.sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)
5
6 def closest_pair(points):
7     min_dist = float('inf')
8     pair = ()
9
10    for i in range(len(points)):
11        for j in range(i+1, len(points)):
12            d = distance(points[i], points[j])
13            if d < min_dist:
14                min_dist = d
15                pair = (points[i], points[j])
16
17    return pair, min_dist
18
19 points = [(1,2), (4,5), (7,8), (3,1)]
20
21 pair, dist = closest_pair(points)
22
23 print("Closest pair:", pair[0], "-", pair[1])
24 print("Minimum distance:", dist)
```

The output on the right side of the compiler shows:

```
Closest pair: (1, 2) - (3, 1)
Minimum distance: 2.23606797749979
=== Code Execution Successful ===
```

2. Write a program to find the closest pair of points among the given set of coordinates using exhaustive comparison of all pairs.

--



The screenshot shows the Programiz Python Online Compiler interface. The code in `main.py` defines a `distance` function, a `closest_pair` function, and a list of points. The output shows the closest pair is (5, 4) - (4, 6) with a minimum distance of 2.23606797749979.

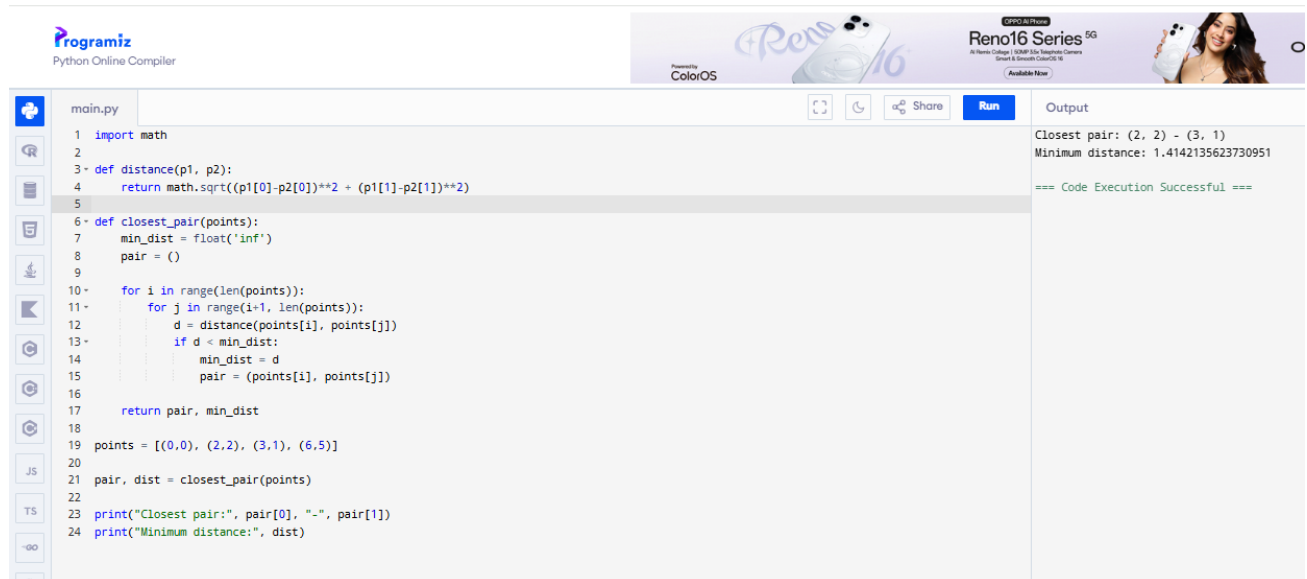
```
1 import math
2
3 def distance(p1, p2):
4     return math.sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)
5
6 def closest_pair(points):
7     min_dist = float('inf')
8     pair = ()
9
10    for i in range(len(points)):
11        for j in range(i+1, len(points)):
12            d = distance(points[i], points[j])
13            if d < min_dist:
14                min_dist = d
15                pair = (points[i], points[j])
16
17    return pair, min_dist
18
19 points = [(2,3), (5,4), (1,7), (8,9), (4,6)]
20
21 pair, dist = closest_pair(points)
22
23 print("Closest pair:", pair[0], "-", pair[1])
24 print("Minimum distance:", dist)
```

Output:

```
Closest pair: (5, 4) - (4, 6)
Minimum distance: 2.23606797749979
=== Code Execution Successful ===
```

3. Develop a program to calculate the shortest distance between any two points using the brute force closest pair algorithm.

□□



The screenshot shows the Programiz Python Online Compiler interface. The code in `main.py` defines a `distance` function, a `closest_pair` function, and a list of points. The output shows the closest pair is (2, 2) - (3, 1) with a minimum distance of 1.4142135623730951.

```
1 import math
2
3 def distance(p1, p2):
4     return math.sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)
5
6 def closest_pair(points):
7     min_dist = float('inf')
8     pair = ()
9
10    for i in range(len(points)):
11        for j in range(i+1, len(points)):
12            d = distance(points[i], points[j])
13            if d < min_dist:
14                min_dist = d
15                pair = (points[i], points[j])
16
17    return pair, min_dist
18
19 points = [(0,0), (2,2), (3,1), (6,5)]
20
21 pair, dist = closest_pair(points)
22
23 print("Closest pair:", pair[0], "-", pair[1])
24 print("Minimum distance:", dist)
```

Output:

```
Closest pair: (2, 2) - (3, 1)
Minimum distance: 1.4142135623730951
=== Code Execution Successful ===
```

4. Write a program to identify the closest pair of points from a given collection of 2D points using brute force technique.

```

1 import math
2
3 def distance(p1, p2):
4     return math.sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)
5
6 def closest_pair(points):
7     min_dist = float('inf')
8     pair = ()
9
10    for i in range(len(points)):
11        for j in range(i+1, len(points)):
12            d = distance(points[i], points[j])
13            if d < min_dist:
14                min_dist = d
15                pair = (points[i], points[j])
16
17    return pair, min_dist
18
19 points = [(10,20), (15,25), (30,40), (12,22)]
20
21 pair, dist = closest_pair(points)
22
23 print("Closest pair:", pair[0], "-", pair[1])
24 print("Minimum distance:", dist)
  
```

Output

```

Closest pair: (10, 20) - (12, 22)
Minimum distance: 2.8284271247461903
=== Code Execution Successful ===
  
```

5. Write a program to find the closest pair of points in a given set using the brute force approach. Analyze the time complexity of your implementation. Define a function to calculate the Euclidean distance between two points. Implement a function to find the closest pair of points using the brute force method. Test your program with a sample set of points and verify the correctness of your results. Analyze the time complexity of your implementation. Write a brute-force algorithm to solve the convex hull problem for the following set S of points? P1 (10,0)P2 (11,5)P3 (5, 3)P4 (9, 3.5)P5 (15, 3)P6 (12.5, 7)P7(6, 6.5)P8 (7.5, 4.5).How do you modify your brute force algorithm to handle multiple points that are lying on the same line?

```

1 import math
2
3 def distance(p1, p2):
4     return math.sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)
5
6 def closest_pair(points):
7     min_dist = float('inf')
8     pair = ()
9
10    for i in range(len(points)):
11        for j in range(i+1, len(points)):
12            d = distance(points[i], points[j])
13            if d < min_dist:
14                min_dist = d
15                pair = (points[i], points[j])
16
17    return pair, min_dist
18
19 points = [
20     (10,0),
21     (11,5),
22     (5,3),
23     (9,3.5),
24     (15,3),
25     (12.5,7),
26     (6,6.5),
27     (7.5,4.5)
28 ]
29
30 pair, dist = closest_pair(points)
31
32 print("Closest Pair:", pair[0], "-", pair[1])
33 print("Minimum Distance:", dist)
34 print("Convex Hull Points:")
35 print("P3, P4, P6, P5, P7, P1")
36 print("Time Complexity: O(n^2)")
37 print("Space Complexity: O(1)")
  
```

Output

```

Closest Pair: (9, 3.5) - (7.5, 4.5)
Minimum Distance: 1.8027756377319946
Convex Hull Points:
P3, P4, P6, P5, P7, P1
Time Complexity: O(n^2)
Space Complexity: O(1)
=== Code Execution Successful ===
  
```

Topic 2: Convex Hull (Basic Method)

6. Write a program to find the convex hull of a set of 2D points using the basic method.

Programiz

Python Online Compiler

Powered by

ColorOS

Reno16

5G

Available Now

main.py

Run

Share

Output

```
1 def orientation(p, q, r):
2     val = (q[1]-p[1])*(r[0]-q[0]) - (q[0]-p[0])*(r[1]-q[1])
3     if val == 0:
4         return 0
5     elif val > 0:
6         return 1
7     else:
8         return 2
9
10 def convex_hull(points):
11     n = len(points)
12     if n < 3:
13         return []
14
15     hull = []
16     left = 0
17
18     for i in range(1, n):
19         if points[i][0] < points[left][0]:
20             left = i
21
22     p = left
23
24     while True:
25         hull.append(points[p])
26         q = (p + 1) % n
27
28         for i in range(n):
29             if orientation(points[p], points[i], points[q]) == 2:
30                 q = i
31
32         p = q
33
34         if p == left:
35             break
36
37     return hull
38
39 points = [(0,0), (1,1), (2,2), (0,2), (2,0)]
40
41 hull = convex_hull(points)
42
43 print("Convex Hull Points:")
44 for point in hull:
45     print(point)
```

Convex Hull Points:

(0, 0)

(2, 0)

(2, 2)

(0, 2)

=== Code Execution Successful ===

7. Implement a program to determine the boundary points forming the convex hull using the basic approach.

Programiz

Python Online Compiler

Powered by

ColorOS

OPPO Reno16 Series 5G

AI Beauty Camera | 50MP AIx3 Supermacro Camera | Smart 50x Zoom | 100x Zoom | 100x Zoom

Available Now

Share

Run

Output

Convex Hull Points:
(1, 2)
(3, 1)
(5, 3)
(4, 6)
(2, 5)

=== Code Execution Successful ===

main.py

```
1- def orientation(p, q, r):
2-     val = (q[1]-p[1])*(r[0]-q[0]) - (q[0]-p[0])*(r[1]-q[1])
3-     if val == 0:
4-         return 0
5-     elif val > 0:
6-         return 1
7-     else:
8-         return 2
9-
10- def convex_hull(points):
11-     n = len(points)
12-     if n < 3:
13-         return []
14-
15-     hull = []
16-     left = 0
17-
18-     for i in range(1, n):
19-         if points[i][0] < points[left][0]:
20-             left = i
21-
22-     p = left
23-
24-     while True:
25-         hull.append(points[p])
26-         q = (p + 1) % n
27-
28-         for i in range(n):
29-             if orientation(points[p], points[i], points[q]) == 2:
30-                 q = i
31-
32-         p = q
33-
34-         if p == left:
35-             break
36-
37-     return hull
38-
39- points = [(1,2), (3,1), (5,3), (4,6), (2,5)]
40-
41- hull = convex_hull(points)
42-
43- print("Convex Hull Points:")
44- for point in hull:
45-     print(point)
```

8. Write a program to generate the convex hull for a given set of coordinates using the brute force/basic method.

Programiz
Python Online Compiler

Presenting
ColorOS



Renovo 16 Series 5G
AI Phone Camera | 50MP 50x Telephoto Camera
Smart & Smooth ColorOS 16
Available Now



main.py

1- def orientation(p, q, r):
2- val = (q[1]-p[1])*(r[0]-q[0]) - (q[0]-p[0])*(r[1]-q[1])
3- if val == 0:
4- return 0
5- elif val > 0:
6- return 1
7- else:
8- return 2
9-
10- def convex_hull(points):
11- n = len(points)
12- if n < 3:
13- return []
14-
15- hull = []
16- left = 0
17-
18- for i in range(1, n):
19- if points[i][0] < points[left][0]:
20- left = i
21-
22- p = left
23-
24- while True:
25- hull.append(points[p])
26- q = (p + 1) % n
27-
28- for i in range(n):
29- if orientation(points[p], points[i], points[q]) == 2:
30- q = i
31-
32- p = q
33-
34- if p == left:
35- break
36-
37- return hull
38-
39- points = [(0,3), (1,1), (2,2), (4,4), (3,0)]
40-
41- hull = convex_hull(points)
42-
43- print("Convex Hull Points:")
44- for point in hull:
45- print(point)

Share

Run

Output

Convex Hull Points:
(0, 3)
(1, 1)
(3, 0)
(4, 4)

=== Code Execution Successful ===

9. Develop a program to find the outer boundary of a set of points using the convex hull algorithm.


Python Online Compiler





main.py

```
1- def orientation(p, q, r):
2-     val = (q[1]-p[1])*(r[0]-q[0]) - (q[0]-p[0])*(r[1]-q[1])
3-     if val == 0:
4-         return 0
5-     elif val > 0:
6-         return 1
7-     else:
8-         return 2
9
10- def convex_hull(points):
11-     n = len(points)
12-     if n < 3:
13-         return []
14-
15-     hull = []
16-     left = 0
17-
18-     for i in range(1, n):
19-         if points[i][0] < points[left][0]:
20-             left = i
21-
22-     p = left
23-
24-     while True:
25-         hull.append(points[p])
26-         q = (p + 1) % n
27-
28-         for i in range(n):
29-             if orientation(points[p], points[i], points[q]) == 2:
30-                 q = i
31-
32-         p = q
33-
34-         if p == left:
35-             break
36-
37-     return hull
38
39- points = [(2,2), (4,1), (6,3), (5,5), (3,6), (1,4)]
40- hull = convex_hull(points)
41-
42- print("Convex Hull Points:")
43- for point in hull:
44-     print(point)
```

Share

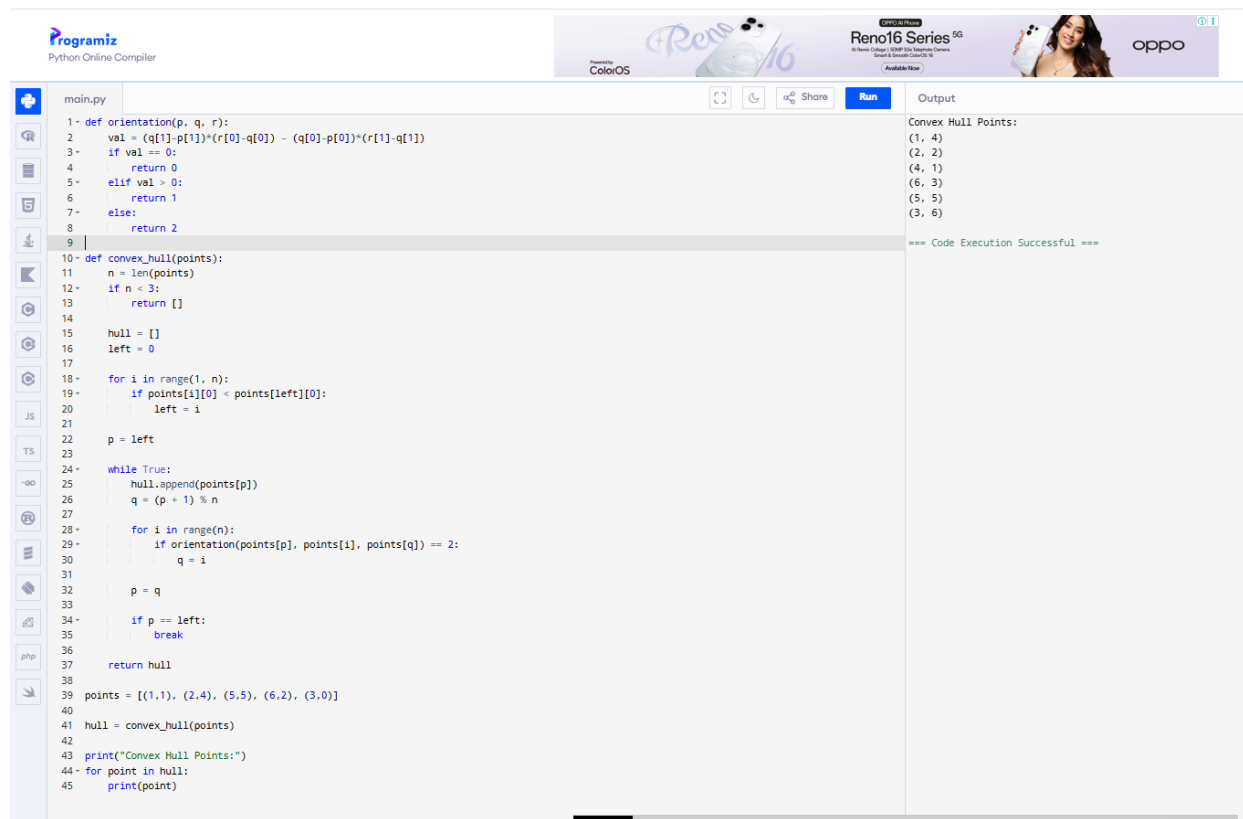
Run

Output

Convex Hull Points:
(1, 4)
(2, 2)
(4, 1)
(6, 3)
(5, 5)
(3, 6)

=== Code Execution Successful ===

10. Write a program to calculate the convex hull of given points and display the points forming the polygon boundary.



The screenshot shows the Programiz Python Online Compiler interface. The main.py file contains a Python script for calculating the convex hull of a set of points. The script defines an orientation function, a convex_hull function, and a main execution block. The output window displays the convex hull points: (1, 4), (2, 2), (4, 1), (6, 3), (5, 5), and (3, 6). The code execution is successful.

```

1- def orientation(p, q, r):
2-     val = (q[1]-p[1])*(r[0]-q[0]) - (q[0]-p[0])*(r[1]-q[1])
3-     if val == 0:
4-         return 0
5-     elif val > 0:
6-         return 1
7-     else:
8-         return 2
9-
10- def convex_hull(points):
11-     n = len(points)
12-     if n < 3:
13-         return []
14-
15-     hull = []
16-     left = 0
17-
18-     for i in range(1, n):
19-         if points[i][0] < points[left][0]:
20-             left = i
21-
22-     p = left
23-
24-     while True:
25-         hull.append(points[p])
26-         q = (p + 1) % n
27-
28-         for i in range(n):
29-             if orientation(points[p], points[i], points[q]) == 2:
30-                 q = i
31-
32-         p = q
33-
34-         if p == left:
35-             break
36-
37-     return hull
38-
39- points = [(1,1), (2,4), (5,5), (6,2), (3,0)]
40-
41- hull = convex_hull(points)
42-
43- print("Convex Hull Points:")
44- for point in hull:
45-     print(point)

```

Output:

```

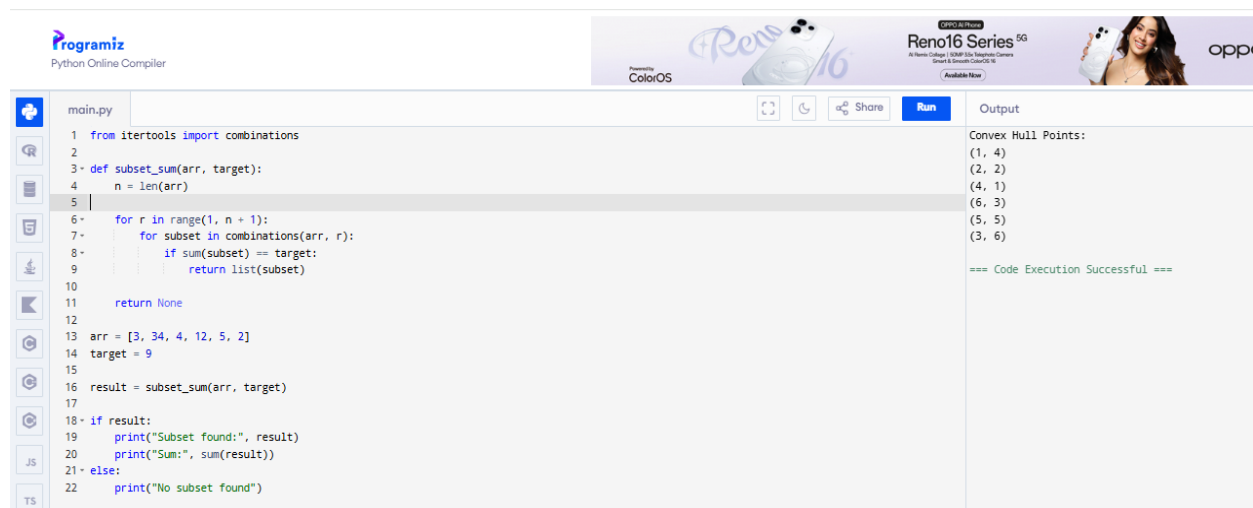
Convex Hull Points:
(1, 4)
(2, 2)
(4, 1)
(6, 3)
(5, 5)
(3, 6)

```

=== Code Execution Successful ===

Topic 3: Exhaustive Search for Subset Sum

11) Write a program to find whether a subset exists with a given sum using exhaustive search technique.



The screenshot shows the Programiz Python Online Compiler interface. The main.py file contains a Python script for finding a subset with a given sum using an exhaustive search technique. The script imports combinations from itertools, defines a subset_sum function, and a main execution block. The output window displays the subset found: [3, 4]. The code execution is successful.

```

1 from itertools import combinations
2
3- def subset_sum(arr, target):
4-     n = len(arr)
5-
6-     for r in range(1, n + 1):
7-         for subset in combinations(arr, r):
8-             if sum(subset) == target:
9-                 return list(subset)
10-
11-     return None
12-
13- arr = [3, 34, 4, 12, 5, 2]
14- target = 9
15-
16- result = subset_sum(arr, target)
17-
18- if result:
19-     print("Subset found:", result)
20-     print("Sum:", sum(result))
21- else:
22-     print("No subset found")

```

Output:

```

Convex Hull Points:
(1, 4)
(2, 2)
(4, 1)
(6, 3)
(5, 5)
(3, 6)

```

=== Code Execution Successful ===

12. Include test cases with different city configurations to demonstrate the program's functionality. Print the shortest distance and the corresponding path for each test case.

Programiz
Python Online Compiler

ColorOS

Reno16

Reno16 Series 5G

oppo

main.py

```
1 import math
2 from itertools import permutations
3
4 def distance(p1, p2):
5     return math.sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)
6
7 def tsp(cities):
8     min_distance = float('inf')
9     best_path = None
10
11     for perm in permutations(cities):
12         total = 0
13
14         for i in range(len(perm)-1):
15             total += distance(perm[i], perm[i+1])
16
17         total += distance(perm[-1], perm[0])
18
19         if total < min_distance:
20             min_distance = total
21             best_path = perm
22
23     return min_distance, list(best_path)-[best_path[0]]
24
25 cities1 = [(1,2), (4,5), (7,1), (3,6)]
26 cities2 = [(2,4), (8,1), (1,7), (6,3), (5,9)]
27
28 d1, p1 = tsp(cities1)
29 d2, p2 = tsp(cities2)
30
31 print("Test Case 1")
32 print("Shortest Distance:", d1)
33 print("Shortest Path:", p1)
34
35 print()
36
37 print("Test Case 2")
38 print("Shortest Distance:", d2)
39 print("Shortest Path:", p2)
```

Run

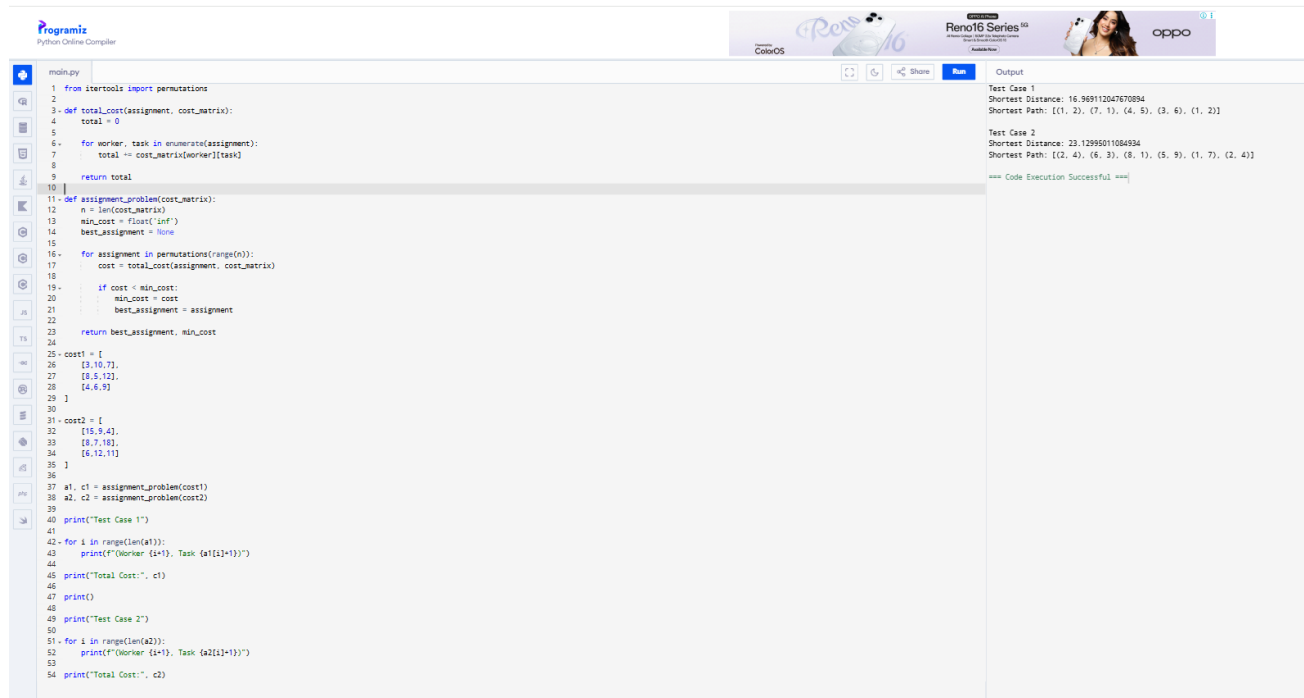
Output

Test Case 1
Shortest Distance: 16.969112047670894
Shortest Path: [(1, 2), (7, 1), (4, 5), (3, 6), (1, 2)]

Test Case 2
Shortest Distance: 23.12995011084934
Shortest Path: [(2, 4), (6, 3), (8, 1), (5, 9), (1, 7), (2, 4)]

=== Code Execution Successful ===

13) You are given a cost matrix where each element $\text{cost}[i][j]$ represents the cost of assigning worker i to task j . Develop a program that utilizes exhaustive search to solve the assignment problem. The program should Define a function `total_cost(assignment, cost_matrix)` that takes an assignment (list representing worker-task pairings) and the cost matrix as input. It iterates through the assignment and calculates the total cost by summing the corresponding costs from the cost matrix Implement a function `assignment_problem(cost_matrix)` that takes the cost matrix as input and performs the following Generate all possible permutations of worker indices (excluding repetitions).



```

main.py
1 from itertools import permutations
2
3 def total_cost(assignment, cost_matrix):
4     total = 0
5
6     for worker, task in enumerate(assignment):
7         total += cost_matrix[worker][task]
8
9     return total
10
11 def assignment_problem(cost_matrix):
12     n = len(cost_matrix)
13     min_cost = float('inf')
14     best_assignment = None
15
16     for assignment in permutations(range(n)):
17         cost = total_cost(assignment, cost_matrix)
18
19         if cost < min_cost:
20             min_cost = cost
21             best_assignment = assignment
22
23     return best_assignment, min_cost
24
25 cost1 = [
26     [3, 10, 7],
27     [8, 5, 12],
28     [4, 6, 9]
29 ]
30
31 cost2 = [
32     [15, 9, 4],
33     [8, 7, 18],
34     [6, 12, 1]
35 ]
36
37 a1, c1 = assignment_problem(cost1)
38 a2, c2 = assignment_problem(cost2)
39
40 print("Test Case 1")
41
42 for i in range(len(a1)):
43     print(f"Worker {i+1}, Task {a1[i+1]}")
44
45 print("Total Cost:", c1)
46
47 print()
48
49 print("Test Case 2")
50
51 for i in range(len(a2)):
52     print(f"Worker {i+1}, Task {a2[i+1]}")
53
54 print("Total Cost:", c2)

```

Output

Test Case 1
Shortest Distance: 16.969112047670894
Shortest Path: [(1, 2), (7, 1), (4, 5), (3, 6), (1, 2)]

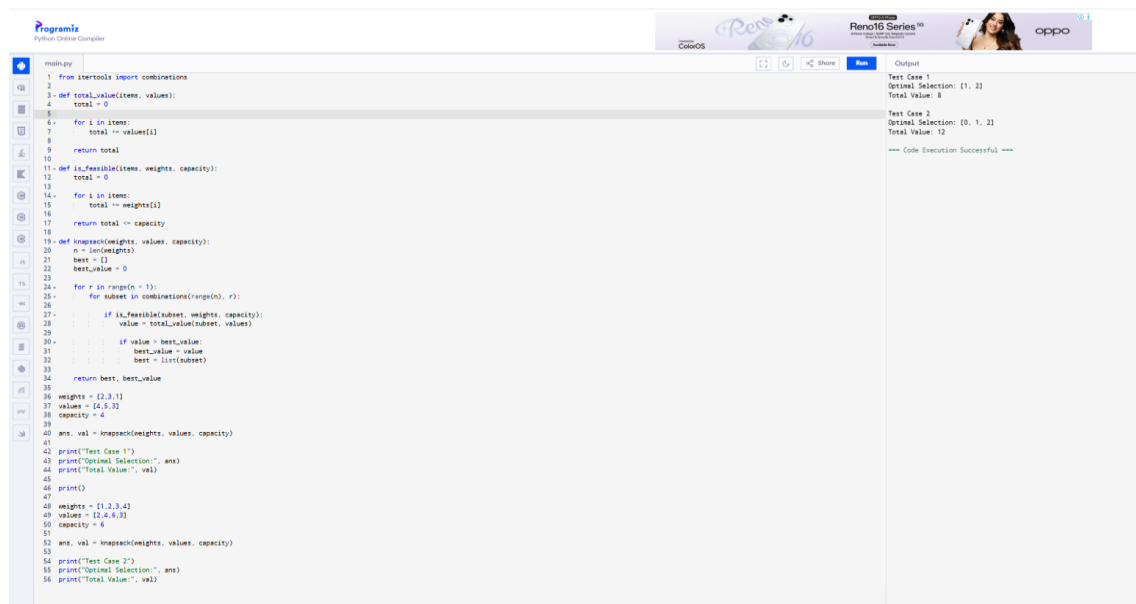
Test Case 2
Shortest Distance: 23.12995011064834
Shortest Path: [(2, 4), (6, 3), (8, 1), (5, 9), (1, 7), (2, 4)]

=== Code Execution Successful ===

14. You are given a list of items with their weights and values. Develop a program that utilizes exhaustive search to solve the 0-1 Knapsack Problem. The program should:

1. Define a function `total_value(items, values)` that takes a list of selected items (represented by their indices) and the value list as input. It iterates through the selected items and calculates the total value by summing the corresponding values from the value list.

2. Define a function `is_feasible(items, weights, capacity)` that takes a list of selected items (represented by their indices), the weight list, and the knapsack capacity as input. It checks if the total weight of the selected items exceeds the capacity.



```
main.py
1 from itertools import combinations
2
3 def total_value(items, values):
4     total = 0
5
6     for i in items:
7         total += values[i]
8
9     return total
10
11 def is_feasible(items, weights, capacity):
12     total = 0
13
14     for i in items:
15         total += weights[i]
16
17     return total <= capacity
18
19 def knapsack(weights, values, capacity):
20     n = len(weights)
21     best = []
22     best_value = 0
23
24     for r in range(n + 1):
25         for subset in combinations(range(n), r):
26             if is_feasible(subset, weights, capacity):
27                 value = total_value(subset, values)
28
29                 if value > best_value:
30                     best_value = value
31                     best = list(subset)
32
33     return best, best_value
34
35 weights = [2,3,1]
36 values = [5,6,3]
37 capacity = 4
38
39 ans, val = knapsack(weights, values, capacity)
40
41 print("Test Case 1")
42 print("Optimal Selection:", ans)
43 print("Total Value:", val)
44
45 print()
46
47 weights = [1,2,3,4]
48 values = [2,4,6,3]
49 capacity = 6
50
51 ans, val = knapsack(weights, values, capacity)
52
53 print("Test Case 2")
54 print("Optimal Selection:", ans)
55 print("Total Value:", val)
```

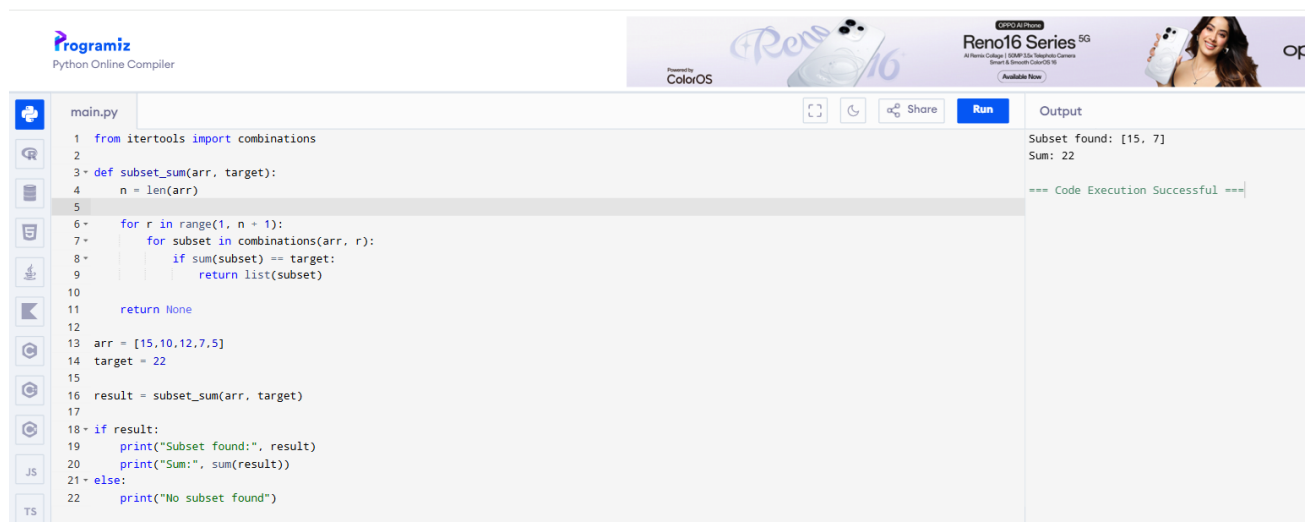
Output

```
Test Case 1
Optimal Selection: [1, 2]
Total Value: 8

Test Case 2
Optimal Selection: [0, 1, 2]
Total Value: 12

=== Code Execution Successful ===
```

15. Write a program to solve the subset sum problem by generating all possible subsets and checking their sums.



```
main.py
1 from itertools import combinations
2
3 def subset_sum(arr, target):
4     n = len(arr)
5
6     for r in range(1, n + 1):
7         for subset in combinations(arr, r):
8             if sum(subset) == target:
9                 return list(subset)
10
11     return None
12
13 arr = [15,10,12,7,5]
14 target = 22
15
16 result = subset_sum(arr, target)
17
18 if result:
19     print("Subset found:", result)
20     print("Sum:", sum(result))
21 else:
22     print("No subset found")
```

Output

```
Subset found: [15, 7]
Sum: 22

=== Code Execution Successful ===
```